

COMPUTER GRAPHICS PROJECT

Bloxorz 3D

Three.js / WebGL puzzle game

Playable puzzle, complete flow

8 official levels with verified par values

rolling cuboid with exact grid rules

fragile tiles, switches, bridges, teleport split mode

keyboard, touch, orbit camera, first-person camera

saved progress and custom level editor

hint system based on shortest-path search

The puzzle state is discrete

state = x, z, orientation

orientation:

standing	one occupied tile
lying_x	two occupied tiles on x axis
lying_z	two occupied tiles on z axis

occupied cells decide legal moves, falling,
switch activation, bridge changes, and winning.

Smooth motion follows exact rules

1. validate the next grid state
2. update the puzzle position and orientation
3. animate the mesh position
4. rotate the mesh with quaternion motion
5. snap back to the exact grid coordinate

This keeps gameplay precise while the 3D motion stays smooth.

Rules on the board

normal tile	supports the block
fragile tile	breaks under a standing block
heavy switch	standing block only
soft switch	any occupied cell
bridge	toggled tile in the level graph
teleport	splits the block into two cubes
merge	joins two adjacent split cubes
goal	accepts only a standing full block

Each mechanic changes move validation.

Course concepts in the scene

geometry	block, tiles, bridges, particles
model transform	position, rotation, scale
view transform	camera frame
projection	perspective screen mapping
rasterization	triangles to fragments
fragment shading	lights and materials
frame buffer	final canvas pixels

RENDERING

Scene and visual feedback

PerspectiveCamera for 3D depth

MeshPhysicalMaterial for light-based surfaces

DirectionalLight, AmbientLight, shadows, fog

SMAA, bloom, and vignette postprocessing

dust, shatter, sparkles, and win particles

procedural audio for movement and switches

optional GPU path tracing with clean fallback

INTERACTION

One game state, multiple controls

keyboard controls for desktop

touch controls for mobile

orbit camera for board inspection

first-person pointer lock camera

HUD for moves, par, level, and hints

localStorage for progress and custom levels

SOLVER

BFS over puzzle states

Breadth-first search explores moves level by level.
The first solution found is the shortest path.

state key:

- x, z, orientation
- bridge state
- split-block state

Used for:

- hint direction
- par verification
- solvability checks

VERIFIED LEVELS

Par values checked by script

1	First Steps	8
2	Over the Edge	7
3	Fragile Ground	6
4	Open Sesame	6
5	Heavy Duty	12
6	Split Decision	8
7	Twin Bridges	13
8	The Gauntlet	15

command:

```
npm run check:levels
```

result:

all official levels solve
with their listed par values

CODE STRUCTURE

Main files

src/main.js	game flow and level lifecycle
src/grid.js	board, tiles, bridges, highlights
src/controls.js	movement, validation, split cubes
src/levels.js	level data, pars, themes
src/solver.js	BFS hint search
src/editor.js	custom level creation
src/scene.js	renderer, camera, lights, postprocessing
src/particles.js	dust, shatter, sparkles, win feedback
scripts/check-levels.js	automated level verification

LIVE DEMO

Five-minute flow

1. level select and saved progress
2. Level 1 movement, move counter, goal
3. fragile tile behavior
4. switch and bridge behavior
5. teleport split and merge
6. hint system
7. editor and first-person camera

SUMMARY

What the project demonstrates

real-time 3D rendering with Three.js/WebGL
model, view, projection, lighting, and shading concepts
event-driven interaction and camera control
state-space search for hints and verification
a complete playable workflow from levels to editor